

bomblab 报告

姓名：杜承霖

学号：2024201507

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	0	0	0	0	0	0

scoreboard 截图：

2024201507	1	0	0	0	0	0	0
------------	---	---	---	---	---	---	---

解题报告

phase_1

```
Someday, I'll conquer the land and have you slain.
```

讲解题目思路

突破点在于调用了一个<strings_not_equal>函数，可以看出是比较字符串，

直接尝试在gdb访问地址x/s 0x3180，得出来0x3180: "Someday, I'll conquer the land and have you slain."

phase_2

```
442214 417866 381998 573752
```

讲解题目思路

phase2主要侧重于循环的嵌套，这里面确实花费了比较多的时间来倒腾的，首先是这里

```
1484: call __isoc99_sscanf@plt
1489: cmp $0x4,%eax
```

告诉我这一关的输入有四个整数,依次存储在栈顶的位置

- **%rdi** 指向矩阵 A (matA) 的首地址。
- **%rsi** 指向矩阵 B (matB) 的首地址。

r11d从零到一，每次循环%rdi增加12个字节（3个int），说明矩阵A有三列
矩阵B是三行二列，A*B得到的矩阵C有两行两列

三个循环变量分别放在了%r11d,%r8d,%eax里面，核心计算操作是这一句

```
`14d8: 8b 14 87      mov    (%rdi,%rax,4),%edx`读取A[i][k]

`14db: 0f af 14 c6      imul   (%rsi,%rax,8),%edx`读取B[k][j]并相乘
```

最后比较输入的四个数是否相等，如果不相等就引爆炸弹

使用GDB查看相关内存地址的内容

```
(gdb) x/6dw 0x6140
```

```
0x6140: 62 846 998
```

```
0x614c: 590 836 617 //MATA
```

```
(gdb) x/6dw 0x6120
```

```
0x6120: 139 536
```

```
0x6128: 102 63
```

```
0x6130: 348 332 //MATB
```

所以答案是442214 417866 381998 573752

phase_3

```
0 922
```

讲解题目思路

核心逻辑是一个基于跳转表的实现的switch语句，开头先用sscanf读取两个整数，第一个数储存在栈顶，第二个数储存在%rsp+4

```
157e: lea 0x1c9b(%rip), %rdx      # 加载跳转表基址 (Jump Table Base)
1585: movslq (%rdx,%rax,4), %rax   # 读取相对偏移量 (Offset)
1589: add %rdx, %rax               # 计算目标绝对地址 (Target Address)
158c: jmp *%rax                   # 执行跳转
```

基址是0x3220，输入的第一个数被用作数组下标

我这里第一个数尝试输入0，发现跳转到了0x1595的代码块

```
1595: mov 0x4b75(%rip), %edx      # 加载全局变量 delta (0x6110)
159b: mov $0x3dd, %eax            # 加载常数 0x3dd (十进制 989)
15a0: sub %edx, %eax               # 计算: eax = 989 - delta
15a2: mov 0x4(%rsp), %edx          # 读取用户输入的第二个值 y
15a6: test %edx, %edx              # 检查 y 是否为负数
...
15ae: cmp %eax, %edx               # 比较 计算值 和 用户输入
```

通过GDB查询内存0x6110的内容，得出delta的值是67

计算出来第二个数是989-67=922

成功拆弹

phase_4

```
31 AB
```

讲解题目思路

通过提示Ancient monks moved sacred disks between poles..., 我们可以猜想phase_4的题目背景是汉诺塔问题(雾)

首先通过调用sscanf函数, 显示他读取了两个值, 一个整数和一个字符串

首先分析一下func4_1:

能看到这个函数在最后又调用了自身, 说明这是个递归函数, 递归的出口是%edi==1, 核心是

```
1666: call 164f          # 递归调用 func4_1(n-1)
166b: lea 0x1(%rax,%rax,1) # 计算 2 * result + 1
```

$f(n) = 2 \cdot f(n-1) + 1$

等价于

$$f(n) = 2^n - 1$$

接下来该分析func4_2:

参数设置%edi=5, %esi=8, %edx='A'(0x41), %ecx='C'(0x43), %r8='B'(0x42),

通过fun1之后f(5)=31, 第一个输入的数字是31

func4_2 是一个分治算法, 用于找出汉诺塔问题的第 x步移动动作。

手动模拟 func4_2(n=5, x=8, Src='A', Dst='C', Aux='B') 的执行流程:

1. 第一层递归 (n=5, x=8):

计算中点 limit = $2^{5-1} - 1 = 15$ 。

比较 x (8) 与 limit (15)。

因为 $8 \leq 15$, 进入左半区递归。

参数变换: 步数 x 不变, 交换 Dst 和 Aux 柱子。

新调用: func4_2(n=4, x=8, Src='A', Dst='B', Aux='C')。

2. 第二层递归 (n=4, x=8):

计算中点 limit = $2^{4-1} - 1 = 7$ 。

比较 x (8) 与 limit (7)。

因为 $8 > 7$, 且正好 $8 = 7 + 1$ 。

命中中间点 (Middle Case): 这是移动当前最大盘子的一步。

动作: 直接将当前的 Src 和 Dst 写入字符串。

当前状态下: Src = 'A', Dst = 'B'。

结果字符串: "AB"

phase_5

aaaggg

讲解题目思路

Phase 5 是一个基于 查表求和 的算法谜题。

输入一个 6 字符的字符串。程序会将每个字符转换为一个索引, 利用该索引在内置的整数数组中查找对应的分值, 并将这 6 个分值累加。最终的总和必须等于 39 (0x27)才能通关。

首先检查输入的字符串的长度，如果不等于6，那么直接跳转爆炸

```
17ed: movzbl (%rax), %edx      # 读取输入字符 (例如 'a')
17f0: and     $0xf, %edx         # 位运算: index = char & 0xF
17f3: add     (%rsi,%rdx,4), %ecx # 查表: sum += array[index]
```

- 1. **提取索引**: 取字符的 ASCII 码，通过 `AND 0xF` 操作保留低 4 位。
- 2. **查表求和**: 以该索引去访问内存地址 `0x3240` 处的数组，将获取的值加到 `%ecx` 中。

```
17ff: cmp     $0x27, %ecx        # 比较 sum 和 39
1802: jne     180d               # 不相等则爆炸
```

循环结束后，累加的总和必须精确等于 39

提取0x3240处的数值

```
(gdb) x/16dw 0x3240
```

得到的映射表如下：

索引 (Low 4 bits)	对应输入字符 (Char)	数组分值 (Value)
0	p (0x70)	2
1	a (0x61)	10
2	b (0x62)	6
3	c (0x63)	1
4	d (0x64)	12
5	e (0x65)	16
6	f (0x66)	9
7	g (0x67)	3
8	h (0x68)	4
9	i (0x69)	7
10	j (0x6a)	14
11	k (0x6b)	5
12	l (0x6c)	11
13	m (0x6d)	8
14	n (0x6e)	15
15	o (0x6f)	13

接下来构造答案就可以了，我这里构造的是aaaggg, 10+10+10+3+3+3=39

phase_6

```
1 3 6 5 4 2
```

讲解题目思路

这份 **Phase 6 逆向分析报告** 详细阐述了如何识别链表结构、提取节点数据以及推导最终排序结果的全过程。你可以直接参考此结构来撰写你的实验报告。

要求输入 6 个整数。这 6 个整数充当了链表节点的索引.重新连接链表中的节点，最后验证重组后的链表节点数值是否按照 降序排列。

```
1836: call read_six_numbers    # 读取6个数
...
184d: (nested loops)          # 双重循环检查重复
1878: cmp $0x5, %eax           # 检查数值范围 (x-1 <= 5u)
```

```
1888: lea 0x4984(%rip), %rdx    # 加载链表头 node1 (0x6220)
188d: mov (%rsp,%rsi,4), %ecx    # 读取用户输入的索引
... (loop to find Nth node) ...
18ac: mov %rdx, 0x20(%rsp,...)   # 将找到的节点地址存入栈数组
```

分析：

程序利用用户输入的数字作为索引，在内存中的链表里查找对应的节点地址。

查找到的 6 个节点地址被依次存储在栈上的指针数组中 (rsp+0x20 开始)。

```
18c5: mov %rax, 0x8(%rbx)        # update next pointer
```

- 程序遍历栈上的指针数组。
- 通过修改每个节点偏移量 0x8 处的值（即 next 指针），将链表在物理内存中按照用户输入的顺序重新连接。

```
1909: mov (%rax), %eax           # 获取下一个节点的值
190b: cmp %eax, (%rbx)           # 比较：当前值 vs 下一个值
190d: jge 18fc                   # 如果 当前 >= 下一个，跳转继续
190f: call explode_bomb          # 否则爆炸
```

程序遍历重组后的链表。

核心校验逻辑是：前一个节点的值必须大于等于后一个节点的值。

需要将链表降序

通过 GDB 查看汇编中引用的链表头地址 <node1> (0x6220)，并顺着 next 指针（偏移量 +8）遍历所有节点。

x/24dw 0x6220
x/2dw 0x6170

提取到的节点数据如下：

节点名称	原始索引 (Index)	节点数值 (Value)	内存地址
Node 1	1	784	0x6220
Node 2	2	128	0x6230
Node 3	3	718	0x6240
Node 4	4	399	0x6250
Node 5	5	481	0x6260
Node 6	6	634	0x6170

Answer: 1 3 6 5 4 2

Secret_phase

ccaac

讲解题目思路

本关本质上是一个“**马踏棋盘**” (Knight's Tour) 变种的迷宫寻路游戏。程序在一个8*8 的内存网格上定义了障碍物（墙壁），要求玩家输入一串字符指令，控制棋子按国际象棋中“马”的走法，从起点 (0, 0) 安全跳跃至终点 (4, 7)。

隐藏关并非默认开启。程序在每次通过普通关卡后，都会调用 `phase_defused` 函数。

```
2101: cml $0x6, num_input_strings # 检查是否通关 Phase 6
...
2176: lea 0x14a4(%rip), %rsi # 加载神秘字符串 "coding"
217d: call strings_not_equal # 检查输入流中是否存在该后缀
```

通过 GDB 调试发现 `%rsi` 指向的字符串为 **"coding"**。程序检测的是 Phase 6输入缓冲区的尾部。因此在 Phase 6的答案后追加了该字符串（coding）以激活隐藏关。

解析fun_7

函数开头在栈上初始化了一个查找表，定义了 8 种可能的移动方向（对应字符的低 3 位索引）：

字符 (Char)	索引	Δx (Col)	Δy (Row)	移动描述
a	1	-1	+2	左 1 下 2
b	2	+1	+2	右 1 下 2

字符 (Char)	索引	Δx (Col)	Δy (Row)	移动描述
c	3	+2	+1	右 2 下 1
d	4	+2	-1	右 2 上 1
...	...	...	...	...
p (0x70)	0	-2	+1	左 2 下 1

每一步移动后，程序检查坐标 (x, y) 是否在 0-7 范围内。

```
1b0f: cmpb $0x1, (%rsi,%rdx,1) # 检查 Grid[y][x] 是否为 1
```

程序访问内存中的 row 数组。如果对应位置的值不为 0，则判定为撞墙，递归返回失败 (0)。

```
1a56: cmp $0x4, %esi # Check x == 4
1a5b: cmp $0x7, %edx # Check y == 7
```

如果到达坐标 (4, 7) 且输入字符串恰好读完，返回成功 (1)。

内存地图提取

. 代表 0x00 安全，# 代表非零 障碍)

```

    0 1 2 3 4 5 6 7  (Col / x轴)
Row 0 . . # . . # . .
Row 1 . . . # . . . #
Row 2 # . # . . # . .
Row 3 # . . . . . . .
Row 4 . # . . # . # .
Row 5 # . . # # . . .
Row 6 . . . . . # . #
Row 7 # # . . # . . . (Node1 Alias)
```

我们需要找到一条从 (0,0) 到 (4,7) 的路径，避开所有 #。

1. Start: (0, 0)
2. 输入 c (2, 1)。此点为 0，安全。
3. 输入 c (4, 2)。此点为 0，安全。
4. 输入 a (3, 4)。此点为 0，安全。
5. 输入 a(2, 6)。此点为 0，安全。
6. 输入 c (4, 7)。抵达终点

反馈/收获/感悟/总结

有趣的地方：进行炸弹的无害化处理，参考的PKU的csapp的笔记，在gdbinit文件里面加入调试的信息，在要调用炸弹爆炸的借点钱插入断点并且跳过传输的函数，这样就做到了炸弹的无害化处理。

难度：10，（满分十分），读汇编语言太太太痛苦了！！

花费时间，读phase2就用了一下午+一晚上，还是在ai的帮助下才搞懂实际的意义
我举得最难的不在于读懂汇编的每一行指令在做什么，难点是把每一行的汇编指令合起来，并且抽象出他们的实际的意义是什么，我觉得这是bomblab最难的一点。
举个例子：phase2和phase4分别是矩阵乘法和汉诺塔，这个没有ai的帮助我是真的难以发现。

参考的重要资料

[更适合北大宝宝体质的 Bomb Lab 踩坑记 • Arthals' ink](#)